

Smart Healthcare Management Platform

Database Design & Documentation

Database Architecture Report

Revised & Enhanced Edition — 2026

MySQL InnoDB	Laravel MVC	ACID Compliant
utf8mb4 Unicode	Soft Deletes	Referential Integrity

Prepared for: Graduation Project | DBMS: MySQL 8.x | Backend: Laravel 10 MVC

1. Introduction & DBMS Selection

The database is the backbone of the **Smart Healthcare Management Platform**, ensuring data integrity across doctors, patients, labs, radiology, pharmacy, and reception.

MySQL InnoDB was selected for:

Feature	Technical Benefit
ACID Transactions	Prevents double-bookings; guarantees atomicity during concurrent inserts
Referential Integrity	Engine-level FK enforcement prevents orphaned medical records
utf8mb4 Encoding	Full Unicode support for Arabic (utf8mb4_unicode_ci)
Laravel Compatibility	Seamless Eloquent ORM, migrations, and query builder integration
JOIN Performance	Adaptive buffer pool and index support for complex multi-table queries

2. Applied Corrections & Enhancements

Four issues were identified and resolved in this revised edition:

■ FIX 1 — ERD ↔ Data Dictionary Inconsistency

The ERD used first_name + last_name while the Data Dictionary used a single Name field. Both layers now consistently use first_name VARCHAR(100) and last_name VARCHAR(100).

■ FIX 2 — Payments Table Missing appointment_id FK

Payments were linked only to a patient with no way to trace back to a specific visit. appointment_id BIGINT UNSIGNED FK has been added to the payments table.

■ FIX 3 — No Soft Deletes

Hard-deleting medical records violates healthcare data-retention regulations. A deleted_at TIMESTAMP NULL column has been added to all core tables (patients, doctors, appointments, prescriptions, payments).

■ FIX 4 — Users / Patients / Doctors Relationship Unclear

The users auth table had no FK link to patients or doctors. A user_id BIGINT UNSIGNED FK column has been added to both tables, establishing a clear 1:1 relationship with the Laravel users table.

✓ All 4 Corrections Applied Successfully

The updated schema, Data Dictionary, and Eloquent model relationships are fully documented in the sections that follow.

3. Entity-Relationship Diagram — Revised

The revised ERD unifies field naming from the Data Dictionary. All entities now use **first_name + last_name** consistently, with new FK columns (user_id, appointment_id, deleted_at) reflected throughout.

Core Tables

Table / Entity	Purpose
users	Authentication & role-based access (Admin, Doctor, Receptionist, Lab Tech ...)
patients	Personal profiles — linked to users via user_id FK
doctors	Medical staff profiles — linked to users via user_id FK
departments	Hospital departments (Cardiology, Radiology, Lab, OPD ...)
appointments	Central hub — bridges Patient ↔ Doctor ↔ Department ↔ Payment
diagnoses	Symptoms, diagnosis text, prescription, treatment plan
prescriptions	Electronic Rx routing to pharmacy / lab / radiology
lab_requests / lab_results	Ordered tests and quantitative results
scan_requests / scan_results	Radiology orders and imaging reports
payments	Financial transactions — now linked to both patient and appointment
inventory	Pharmacy & supply stock with expiration tracking

Relationship Summary

From → To	Type	Description
users → patients / doctors	1 : 1	One login account per patient or doctor
patients → appointments	1 : N	One patient, many appointments
doctors → appointments	1 : N	One doctor, many appointments
departments → appointments	1 : N	One department, many appointments
appointments → payments	1 : 1	One appointment, one payment record (NEW FK)
appointments → diagnoses	1 : 1	One appointment, one diagnosis
diagnoses → prescriptions	1 : N	One diagnosis can generate many Rx
prescriptions → lab_requests	1 : N	One Rx may order multiple lab tests
prescriptions → scan_requests	1 : N	One Rx may order multiple scans

4. Data Dictionary — Complete & Revised

4.1 users (Authentication Table)

Central auth table used by Laravel. Every actor has exactly one row here.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Unique user ID
name	VARCHAR(255)	NOT NULL	Display name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email
email_verified_at	TIMESTAMP	NULL	Email verification time
password	VARCHAR(255)	NOT NULL	Hashed (bcrypt)
role	ENUM	NOT NULL, DEFAULT 'patient'	admin doctor receptionist lab_tech radiology_tech pharmacist patient
remember_token	VARCHAR(100)	NULL	Laravel remember-me token
deleted_at	TIMESTAMP	NULL ← NEW	Soft delete timestamp
created_at / updated_at	TIMESTAMP	NULL	Eloquent timestamps

```
CREATE TABLE `users` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `name` VARCHAR(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `email` VARCHAR(255) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `role` ENUM('admin','doctor','receptionist','lab_tech','radiology_tech','pharmacist','patient')  
    NOT NULL DEFAULT 'patient',  
  `deleted_at` TIMESTAMP NULL DEFAULT NULL, -- Soft Delete (NEW)  
  `created_at` TIMESTAMP NULL DEFAULT NULL,  
  `updated_at` TIMESTAMP NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `users_email_unique` (`email`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.2 patients (FIXED)

✓ Corrections: first_name + last_name | user_id FK | deleted_at added

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Patient ID
user_id	BIGINT UNSIGNED	FK → users.id ← NEW	Auth account link
first_name	VARCHAR(100)	NOT NULL ← FIXED	First name (was 'Name')
last_name	VARCHAR(100)	NOT NULL ← FIXED	Last name (was missing)
dob	DATE	NULL	Date of birth
gender	VARCHAR(10)	NULL	Male / Female / Other
phone	VARCHAR(20)	NULL	Contact number

Column	Type	Constraints	Description
address	TEXT	NULL	Residential address
email	VARCHAR(100)	UNIQUE, NULL	Email (Gmail API)
insurance_info	TEXT	NULL	Insurance policy details
medical_history	TEXT	NULL	Chronic illnesses, history
deleted_at	TIMESTAMP	NULL ← NEW	Soft delete timestamp
created_at / updated_at	TIMESTAMP	NULL	Eloquent timestamps

```
CREATE TABLE `patients` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  `user_id` BIGINT UNSIGNED NOT NULL, -- FK (NEW)  
  `first_name` VARCHAR(100) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `last_name` VARCHAR(100) COLLATE utf8mb4_unicode_ci NOT NULL,  
  `dob` DATE NULL,  
  `gender` VARCHAR(10) COLLATE utf8mb4_unicode_ci NULL,  
  `phone` VARCHAR(20) COLLATE utf8mb4_unicode_ci NULL,  
  `address` TEXT COLLATE utf8mb4_unicode_ci NULL,  
  `email` VARCHAR(100) COLLATE utf8mb4_unicode_ci NULL,  
  `insurance_info` TEXT COLLATE utf8mb4_unicode_ci NULL,  
  `medical_history` TEXT COLLATE utf8mb4_unicode_ci NULL,  
  `deleted_at` TIMESTAMP NULL DEFAULT NULL, -- Soft Delete (NEW)  
  `created_at` TIMESTAMP NULL DEFAULT NULL,  
  `updated_at` TIMESTAMP NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `patients_email_unique` (`email`),  
  CONSTRAINT `patients_user_fk`  
    FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON DELETE RESTRICT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.3 doctors (FIXED)

✓ Corrections: first_name + last_name | user_id FK | deleted_at added

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Doctor ID
user_id	BIGINT UNSIGNED	FK → users.id ← NEW	Auth account link
first_name	VARCHAR(100)	NOT NULL ← FIXED	First name
last_name	VARCHAR(100)	NOT NULL ← FIXED	Last name
specialization	VARCHAR(100)	NULL	Medical specialty
phone	VARCHAR(20)	NULL	Contact number
schedule	TEXT	NULL	Weekly availability
availability_status	BOOLEAN	DEFAULT TRUE	Accepting bookings?
deleted_at	TIMESTAMP	NULL ← NEW	Soft delete timestamp
created_at / updated_at	TIMESTAMP	NULL	Eloquent timestamps

4.4 appointments (Central Scheduling Hub)

The dynamic engine of Patient Flow Management — bridges Patient ↔ Doctor ↔ Department ↔ Financial.

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Appointment ID
patient_id	BIGINT UNSIGNED	FK → patients.id, NOT NULL	Patient
doctor_id	BIGINT UNSIGNED	FK → doctors.id, NOT NULL	Doctor
department_id	BIGINT UNSIGNED	FK → departments.id, NULL	Department
appointment_date	DATE	NOT NULL	Calendar date
start_time	TIME	NOT NULL	Slot start
end_time	TIME	NOT NULL	Slot end (prevents overlap)
status	VARCHAR(255)	DEFAULT 'pending'	pending confirmed cancelled completed
queue_number	INT UNSIGNED	NULL	Live queue position
treatment	VARCHAR(255)	NOT NULL	Clinical purpose of visit
notes	TEXT	NULL, utf8mb4	Patient complaints / Arabic supported
fee	DECIMAL(10,2)	NOT NULL DEFAULT 0.00	No rounding errors
phone_number	VARCHAR(255)	NULL	Contact for appointment
checked_in_at	TIMESTAMP	NULL	Arrival → KPI calculation
auto_confirmed_at	TIMESTAMP	NULL	Auto-confirm trigger
confirmation_email_sent_at	TIMESTAMP	NULL	Email tracking
deleted_at	TIMESTAMP	NULL ← NEW	Soft delete timestamp
created_at / updated_at	TIMESTAMP	NULL	Eloquent timestamps

```
CREATE TABLE `appointments` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `patient_id` BIGINT UNSIGNED NOT NULL,
  `doctor_id` BIGINT UNSIGNED NOT NULL,
  `department_id` BIGINT UNSIGNED DEFAULT NULL,
  `appointment_date` DATE NOT NULL,
  `start_time` TIME NOT NULL,
  `end_time` TIME NOT NULL,
  `status` VARCHAR(255) NOT NULL DEFAULT 'pending',
  `queue_number` INT UNSIGNED DEFAULT NULL,
  `treatment` VARCHAR(255) NOT NULL,
  `notes` TEXT COLLATE utf8mb4_unicode_ci,
  `fee` DECIMAL(10,2) NOT NULL DEFAULT '0.00',
  `checked_in_at` TIMESTAMP NULL DEFAULT NULL,
  `deleted_at` TIMESTAMP NULL DEFAULT NULL, -- Soft Delete (NEW)
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
```

```
PRIMARY KEY (`id`),
CONSTRAINT `appt_patient_fk`
    FOREIGN KEY (`patient_id`) REFERENCES `patients`(`id`) ON DELETE RESTRICT,
CONSTRAINT `appt_doctor_fk`
    FOREIGN KEY (`doctor_id`) REFERENCES `doctors`(`id`) ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.5 payments (FIXED — appointment_id Added)

✓ Correction: appointment_id FK now links payment to the specific clinical visit

Column	Type	Constraints	Description
id	BIGINT UNSIGNED	PK, AUTO_INCREMENT	Transaction ID
patient_id	BIGINT UNSIGNED	FK → patients.id	Responsible patient
appointment_id	BIGINT UNSIGNED	FK → appointments.id ← NEW	Linked appointment
amount	DECIMAL(10,2)	NOT NULL	Precise monetary amount
payment_purpose	VARCHAR(150)	NULL	Consultation / Lab / Scan
status	ENUM	NOT NULL	Paid Pending Refunded
transaction_date	TIMESTAMP	NULL	Payment processing time
deleted_at	TIMESTAMP	NULL ← NEW	Soft delete timestamp
created_at / updated_at	TIMESTAMP	NULL	Eloquent timestamps

```
CREATE TABLE `payments` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  `patient_id` BIGINT UNSIGNED NOT NULL,
  `appointment_id` BIGINT UNSIGNED NULL, -- FK (NEW)
  `amount` DECIMAL(10,2) NOT NULL,
  `payment_purpose` VARCHAR(150) COLLATE utf8mb4_unicode_ci NULL,
  `status` ENUM('Paid','Pending','Refunded') NOT NULL DEFAULT 'Pending',
  `transaction_date` TIMESTAMP NULL DEFAULT NULL,
  `deleted_at` TIMESTAMP NULL DEFAULT NULL, -- Soft Delete (NEW)
  `created_at` TIMESTAMP NULL DEFAULT NULL,
  `updated_at` TIMESTAMP NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `payments_patient_fk`
    FOREIGN KEY (`patient_id`) REFERENCES `patients`(`id`) ON DELETE RESTRICT,
  CONSTRAINT `payments_appt_fk`
    FOREIGN KEY (`appointment_id`) REFERENCES `appointments`(`id`) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.6 Remaining Tables — Summary

Table	Key Columns
diagnoses	appointment_id, patient_id, doctor_id, symptoms, diagnosis, prescription, treatment_plan
prescriptions	appointment_id, patient_id, doctor_id, diagnosis_notes, medication, lab_required, radiology_required

Table	Key Columns
lab_requests	diagnosis_id, patient_id, doctor_id, test_name, status, requested_at
lab_results	lab_request_id, patient_id, doctor_id, entered_by_id, result_value, uploaded_file_path
scan_requests	diagnosis_id, patient_id, doctor_id, scan_type, status, requested_at
scan_results	scan_request_id, patient_id, doctor_id, findings, impression, scan_image
inventory	item_name, category, quantity, unit, expiration_date, reorder_level
departments	name, description, head_doctor_id

5. Laravel MVC — Eloquent Model Relationships

All models touching medical data use the **SoftDeletes** trait. The corrections map directly to these relationships:

User Model

```
// app/Models/User.php
use Illuminate\Database\Eloquent\SoftDeletes;
class User extends Authenticatable {
    use SoftDeletes;
    protected $fillable = ['name', 'email', 'password', 'role'];

    public function patient() { return $this->hasOne(Patient::class); }
    public function doctor() { return $this->hasOne(Doctor::class); }
}
```

Patient Model

```
// app/Models/Patient.php
use Illuminate\Database\Eloquent\SoftDeletes;
class Patient extends Model {
    use SoftDeletes;

    public function user() { return $this->belongsTo(User::class); }
    public function appointments() { return $this->hasMany(Appointment::class); }
    public function diagnoses() { return $this->hasMany(Diagnosis::class); }
    public function payments() { return $this->hasMany(Payment::class); }
    public function labResults() { return $this->hasMany(LabResult::class); }
    public function scanResults() { return $this->hasMany(ScanResult::class); }
}
```

Doctor Model

```
// app/Models/Doctor.php
use Illuminate\Database\Eloquent\SoftDeletes;
class Doctor extends Model {
    use SoftDeletes;

    public function user() { return $this->belongsTo(User::class); }
    public function appointments() { return $this->hasMany(Appointment::class); }
    public function diagnoses() { return $this->hasMany(Diagnosis::class); }
}
```

Appointment Model (NEW: payment relationship)

```
// app/Models/Appointment.php
use Illuminate\Database\Eloquent\SoftDeletes;
class Appointment extends Model {
    use SoftDeletes;

    public function patient() { return $this->belongsTo(Patient::class); }
    public function doctor() { return $this->belongsTo(Doctor::class); }
    public function department() { return $this->belongsTo(Department::class); }
    public function diagnosis() { return $this->hasOne(Diagnosis::class); }
    public function payment() { return $this->hasOne(Payment::class); } // NEW
}
```

Payment Model (NEW: appointment relationship)

```
// app/Models/Payment.php
use Illuminate\Database\Eloquent\SoftDeletes;
class Payment extends Model {
    use SoftDeletes;

    public function patient() {
        return $this->belongsTo(Patient::class);
    }
    // NEW – trace payment back to specific appointment
    public function appointment() {
        return $this->belongsTo(Appointment::class);
    }
}
```

Migration Snippets

```
// Migration: add corrections to patients
Schema::table('patients', function (Blueprint $table) {
    $table->unsignedBigInteger('user_id')->after('id');
    $table->foreign('user_id')->references('id')->on('users')
        ->onDelete('restrict');
    $table->softDeletes(); // adds deleted_at TIMESTAMP NULL
});

// Migration: add appointment_id to payments
Schema::table('payments', function (Blueprint $table) {
    $table->unsignedBigInteger('appointment_id')->nullable()->after('patient_id');
    $table->foreign('appointment_id')->references('id')->on('appointments')
        ->onDelete('setNull');
    $table->softDeletes();
});
```

6. Integrity Constraints & Referential Actions

Relationship	ON DELETE	Reason
users → patients / doctors	RESTRICT	Cannot delete user with medical records
patients → appointments	RESTRICT	Medical history must be preserved
doctors → appointments	RESTRICT	Appointment references the doctor
appointments → payments	SET NULL	Payment kept; appointment_id nullable
diagnoses → prescriptions	CASCADE	Rx meaningless without diagnosis
scan_requests → scan_results	CASCADE	Results belong to request
patients → scan_results	CASCADE	Results belong to patient

1 : N Relationships

- **patients** → appointments, diagnoses, payments, lab_results, scan_results
- **doctors** → appointments, diagnoses, prescriptions
- **departments** → appointments
- **diagnoses** → prescriptions, lab_requests, scan_requests

1 : 1 Relationships

- users → patients (via patients.user_id)
- users → doctors (via doctors.user_id)
- appointments → diagnoses
- appointments → payments (NEW)

7. Data Workflow Execution (Patient Journey)

1	Booking Stage Patient registers → users row created → patients row linked via user_id → appointment inserted (status = 'pending').
2	Check-in Stage Receptionist marks arrival → checked_in_at set → queue_number assigned → status = 'confirmed'.
3	Examination Stage Doctor queries patient via unique ID → diagnosis saved → medication, lab orders, scan orders stored.
4	Integration Stage Pharmacy, lab, radiology dashboards query outstanding orders via Eloquent on prescriptions / lab_requests / scan_requests in real time.

5

Financial Update Stage

Lab/radiology uploads results → visible on patient portal → reception marks payment → payments.status = 'Paid', linked via appointment_id.

6

Closure Stage

Appointment status → 'completed' → fee finalised → all records preserved via soft delete.

8. Future Database Enhancements

A	Partitioning & Sharding The appointments table will be horizontally partitioned by appointment_date (one partition per year), reducing query time as records grow to millions.
B	Composite Index Optimization Indexes on (doctor_id, appointment_date) and (patient_id, status) deliver sub-millisecond latency for reception-desk lookups at peak hours.
C	Data Warehousing & Archiving Historical data older than 3 years will auto-migrate to an OLAP Data Warehouse, keeping the production DB lean without losing medical records.
D	Multi-Tenant Architecture Expansion to multiple branches will use Shared Database / Separate Schemas or a branch_id tenant column enforcing absolute data isolation.
E	Master-Slave Replication (HA) Master handles all writes; Slave replicas serve read-heavy operations (reports, patient histories), achieving zero-downtime and disaster recovery.